PERCONA®

PERCONA
for PostgreSQL

Vector Database Performance Challenges
A CaseStudy

©2025 Percona

Case Study Overview

- whoami
- whoareu
- purpose of this presentation
- breakdown

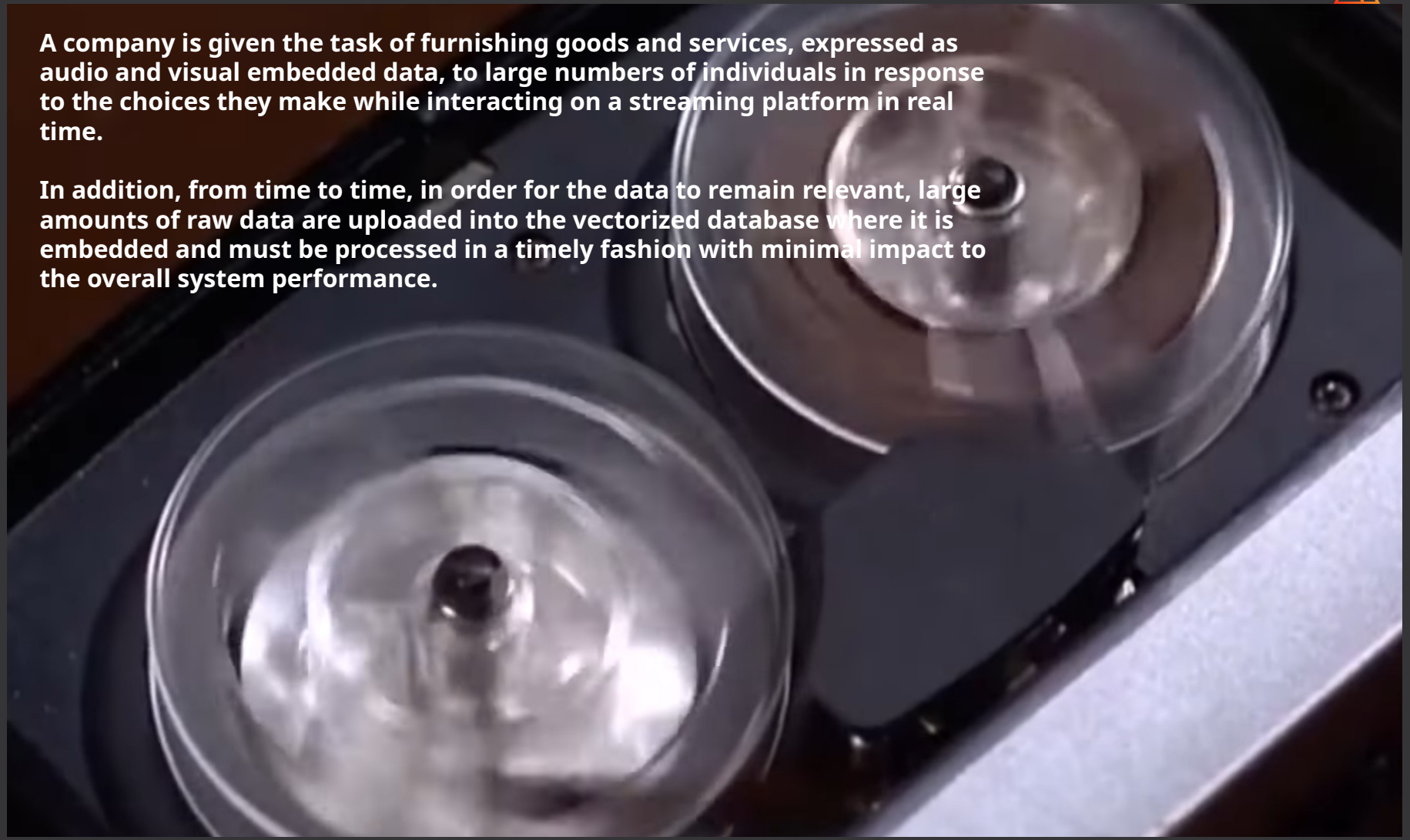
©2025 Percona

Breakdown

- The Mission
- The Task
- About Vector Databases
- PostgreSQL As A Vector Database
- PostgreSQL Vector Indexes
- A/B Testing
- The Table Schema
- Workloads ala pgbench
- Analysis
- Recommendations

©2025 Percona

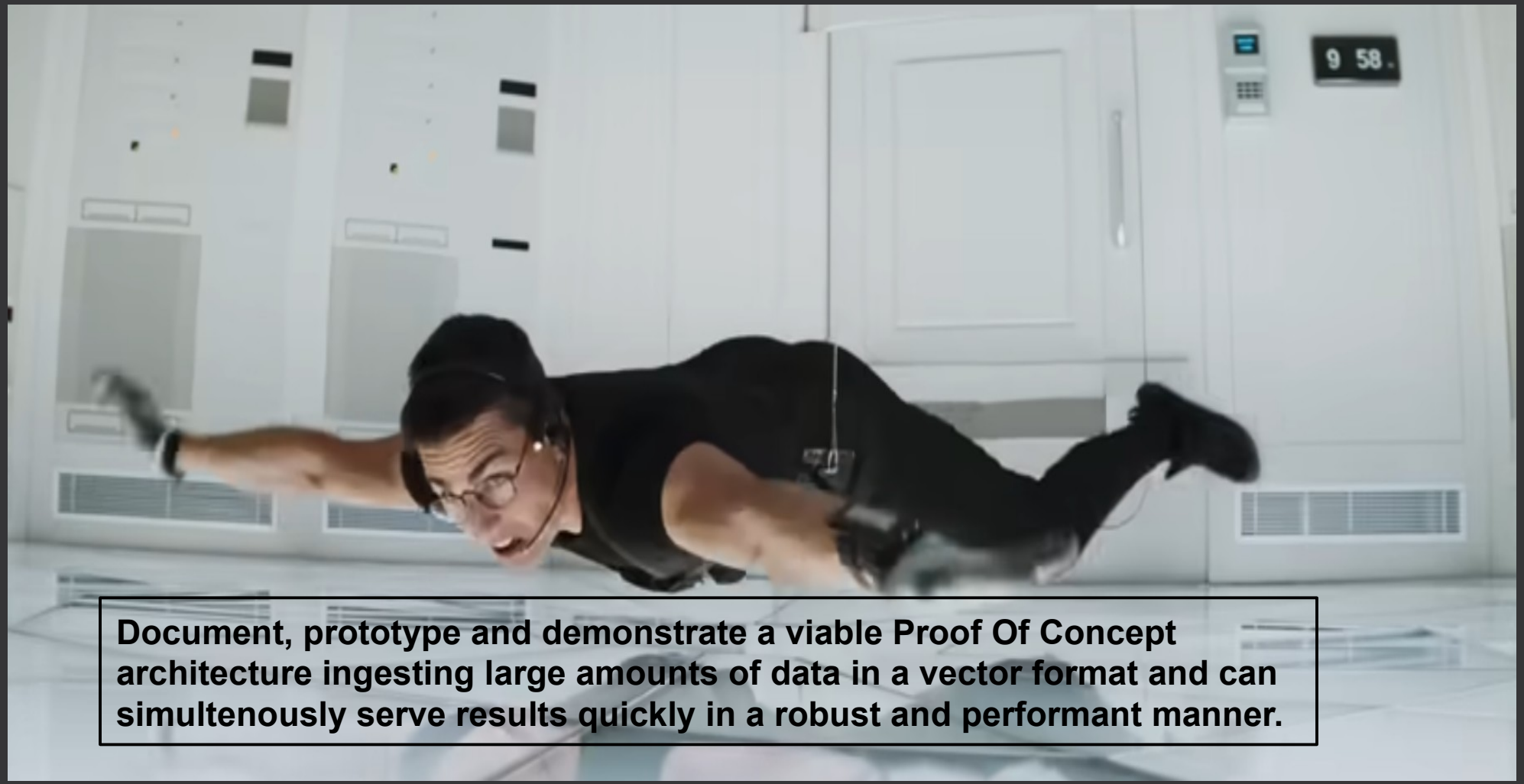
The Mission



A company is given the task of furnishing goods and services, expressed as audio and visual embedded data, to large numbers of individuals in response to the choices they make while interacting on a streaming platform in real time.

In addition, from time to time, in order for the data to remain relevant, large amounts of raw data are uploaded into the vectorized database where it is embedded and must be processed in a timely fashion with minimal impact to the overall system performance.

©2025 Percona



Document, prototype and demonstrate a viable Proof Of Concept architecture ingesting large amounts of data in a vector format and can simultenously serve results quickly in a robust and performant manner.

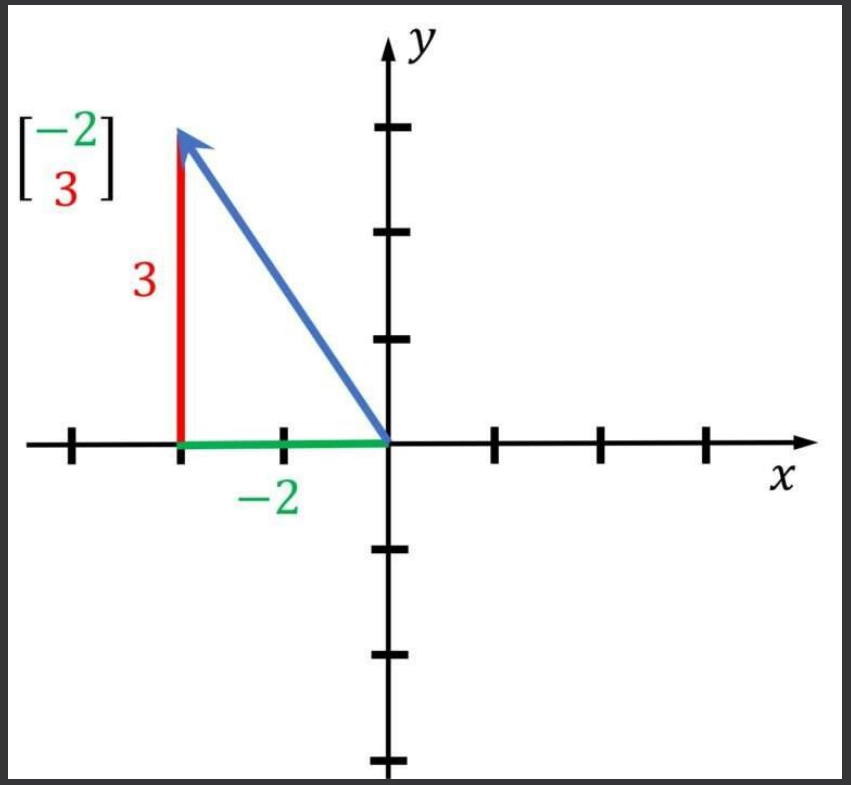
©2025 Percona

About Vector Databases

Everything you ever wanted to ask your mother but was afraid to ask.

Vectors

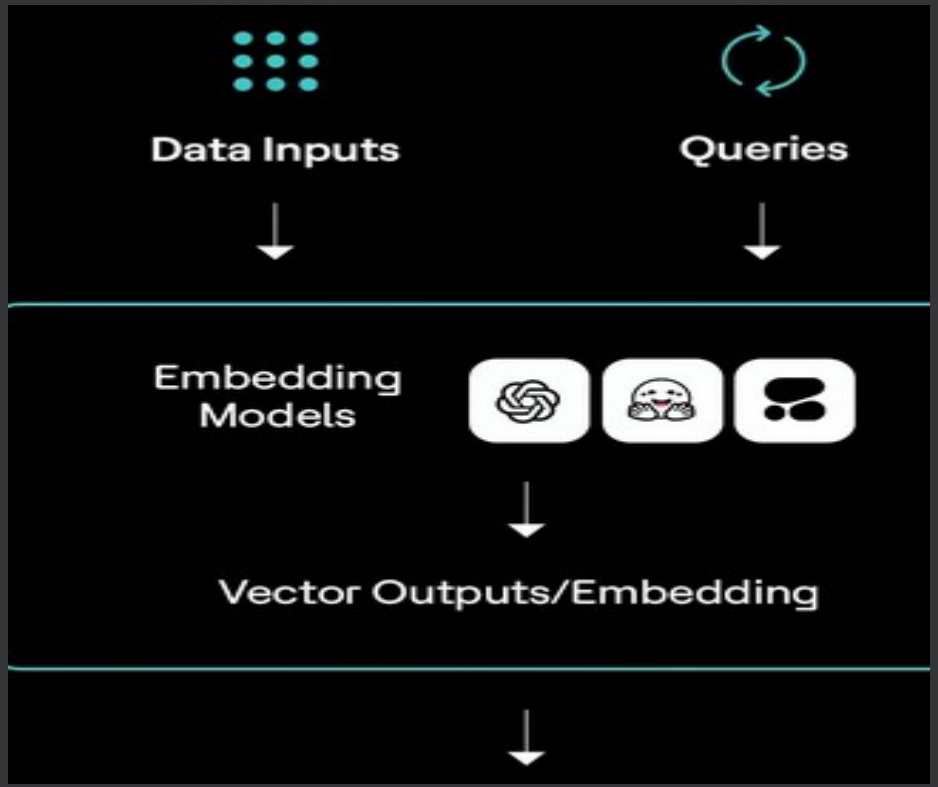
Mathematical objects that represent quantities with both magnitude and direction.



©2025 Percona

Vector Database

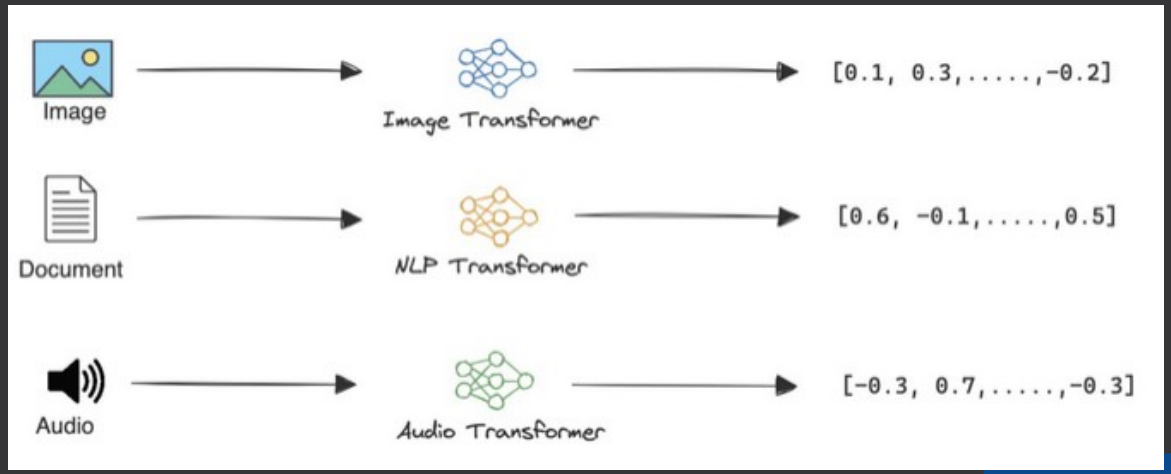
A specialized database that stores, indexes, and searches data as numerical representations, called **embeddings**



©2025 Percona

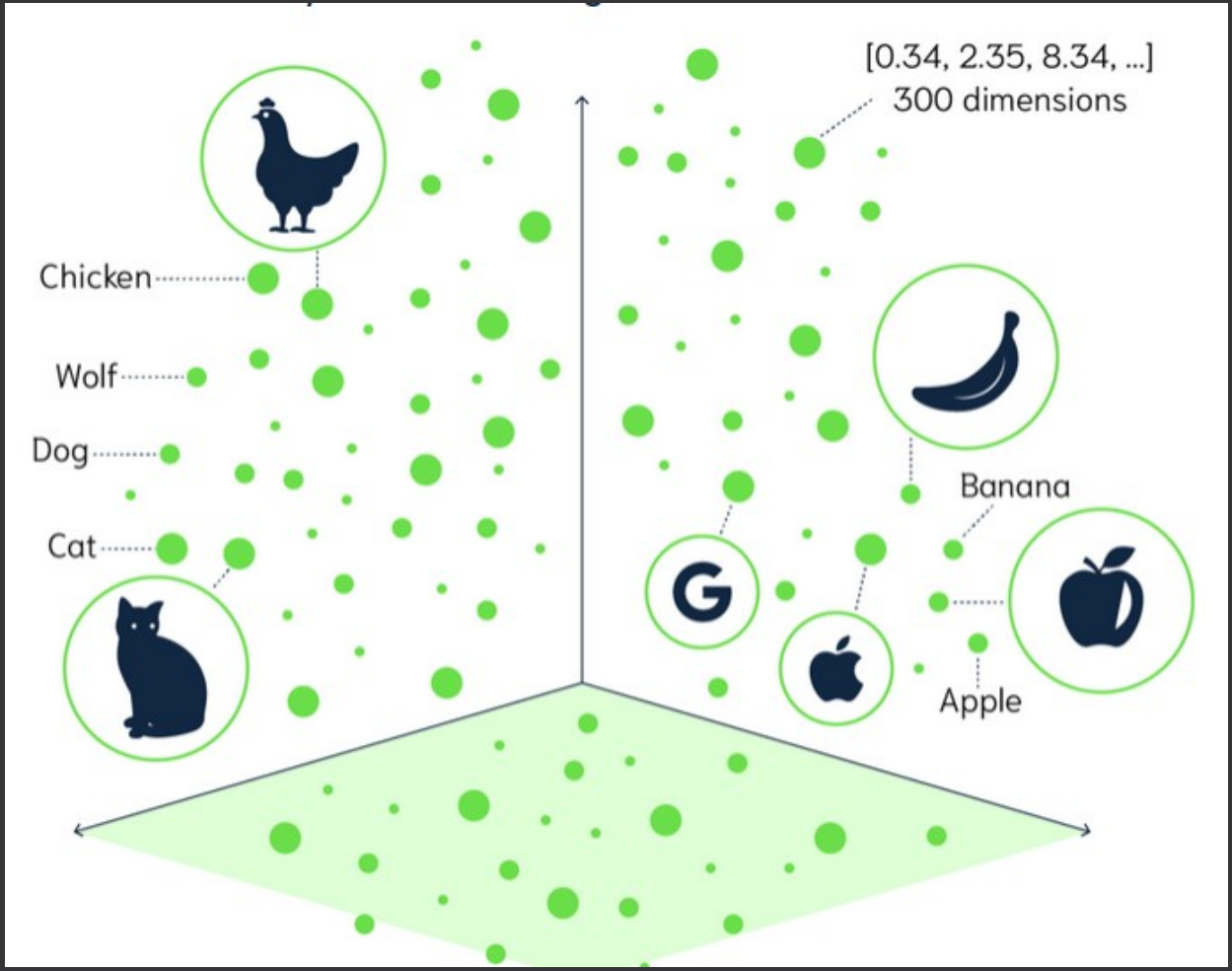
Vector Embedding

- A numerical representation of data as for example words or images in a high-dimensional space.
- It translates complex, non-numeric information into lists of numbers (vectors) that machine learning models can use for tasks like semantic search, recommendation engines, and chatbot responses.
- Implementation:
  - Data is converted by AI models into vectors.
  - The database calculates the distance between these vectors identifying relationships and **semantic similarity.**
  - The vector search engine allows for finding the nearest **neighbors** in a dataset, providing highly relevant results, even if the data does not contain identical keywords.



©2025 Percona

Similarity & Nearest Neighbors



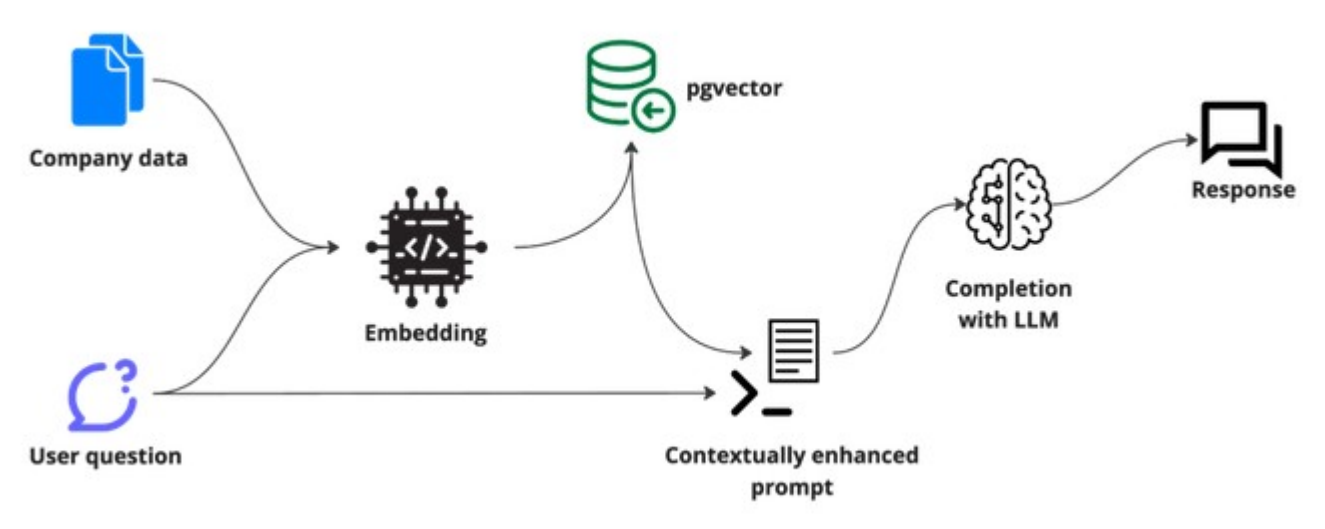
©2025 Percona

PostgreSQL as a Vector Database

PostgreSQL is implemented as a vector database by virtue of vectorized indexes allowing for the storage of embedded data.

Key Attributes

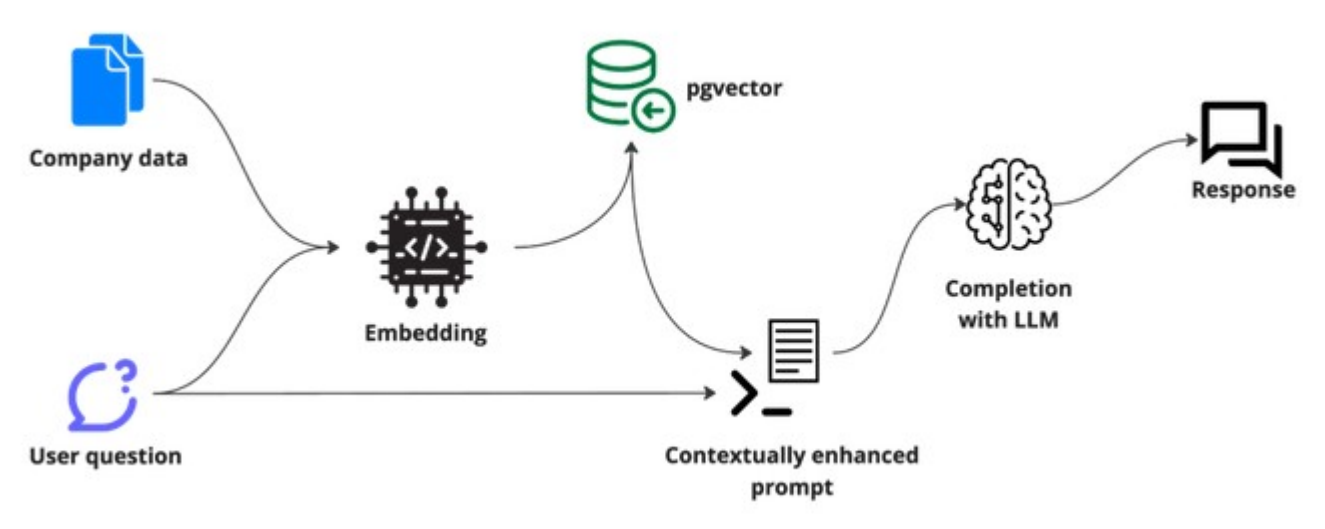
- practical
- cost effective
- ideal for teams already using postgres
- small to medium scale AI applications i.e. <= 10E6 vectors



©2025 Percona

pgvector

- An open-source extension for PostgreSQL
- Enables the storage, indexing, and similarity search of high-dimensional vectors in postgres
- Designed to handle both traditional structured data and modern AI-related vector data



©2025 Percona

pgvectorscale

- An open-source extension for PostgreSQL
- Enables the storage, indexing, and similarity search of high-dimensional vectors in postgres
- Designed to handle both traditional structured data and modern AI-related vector data

An example of how to set the `num_neighbors` parameter is:

```
CREATE INDEX document_embedding_idx ON document_embedding
USING diskann (embedding) WITH(num_neighbors=50);
```

An example of creating an index with label-based filtering:

```
CREATE INDEX document_embedding_idx ON document_embedding
USING diskann (embedding vector_cosine_ops, labels);
```


©2025 Percona

Getting it

- https://github.com/pgvector/pgvector
- https://github.com/timescale/pgvectorscale
- https://github.com/timescale/timescaledb

©2025 Percona

A/B Testing Methodology

1) Prototype the record
2) Prototype the table
3) Determine Partitioning: *range/hash hybrid*
4) Populate the table
5) Assign and configure the index to use
6) Prototype and script benchmarking DML operations: *pgbench*
7) Run workloads
8) Collate and document workload results
9) Analysis & Summary
10) Optimal Architecture criteria:
    1) performance
    2) stability
    3) scaling

©2025 Percona

The Record, JSON


```
{
    "filetype": 39,
    "version": 0,
    "flags": 2,
    "guid": 36028797190789813,
    "record_count": 85114,
    "magic": 3400596190,
    "embed": 128,
    "points":
    [
        [
            36028797190789813,
            1733127300450,
            "[159,122,102,144,106,83,67,61,51,0,0,0,56,56,55,73,68,63,138,102,79,97,89,79,11,19,18,31,31,31,69,76
            ,76,58,49,41,7,16,17,8,13,12,71,71,71,18,18,12,8,10,10,53,33,15,18,28,27,0,0,0,0,0,0,0,0,0,0,0,0,0,0,
            0,0,0,0]"
        ]
    ]
}
```


©2025 Percona

The Tables

```
create table content_frames_video (
     id bigint generated always as identity
    ,u_id bigint not null
    ,ts bigint not null
    ,frame vector(75) not null
    ,t_stamp timestamptz not null default now()
);
```

```
create table content_frames_audio (
     id bigint generated always as identity
    ,u_id bigint not null
    ,ts bigint not null
    ,frame vector(35) not null
    ,t_stamp timestamptz not null default now()
);
```


©2025 Percona

Partitioning Regime

1) Create Hyper Tables
2) Partition: Range/Hash
3) Chunks: 6h, 48h
4) Data Population

©2025 Percona

Timescale Partitions

Create Hyper Tables:

```
-- create hypertables
select create_hypertable('content_frames_video','t_stamp');
select create_hypertable('content_frames_audio','t_stamp');
```


©2025 Percona

Timescale Partitions

Configuring CHUNKS:

In order to better identity optimization opportunities, two distinct databases were provisioned for benchmarking using two different CHUNK sizes.

6h CHUNK sized partitions: (300+partitions)

```
select * from set_chunk_time_interval('content_frames_video',interval '6 hours');
select * from set_chunk_time_interval('content_frames_audio',interval '6 hours');
```

48h CHUNK sized partitions: (<45) partitions)

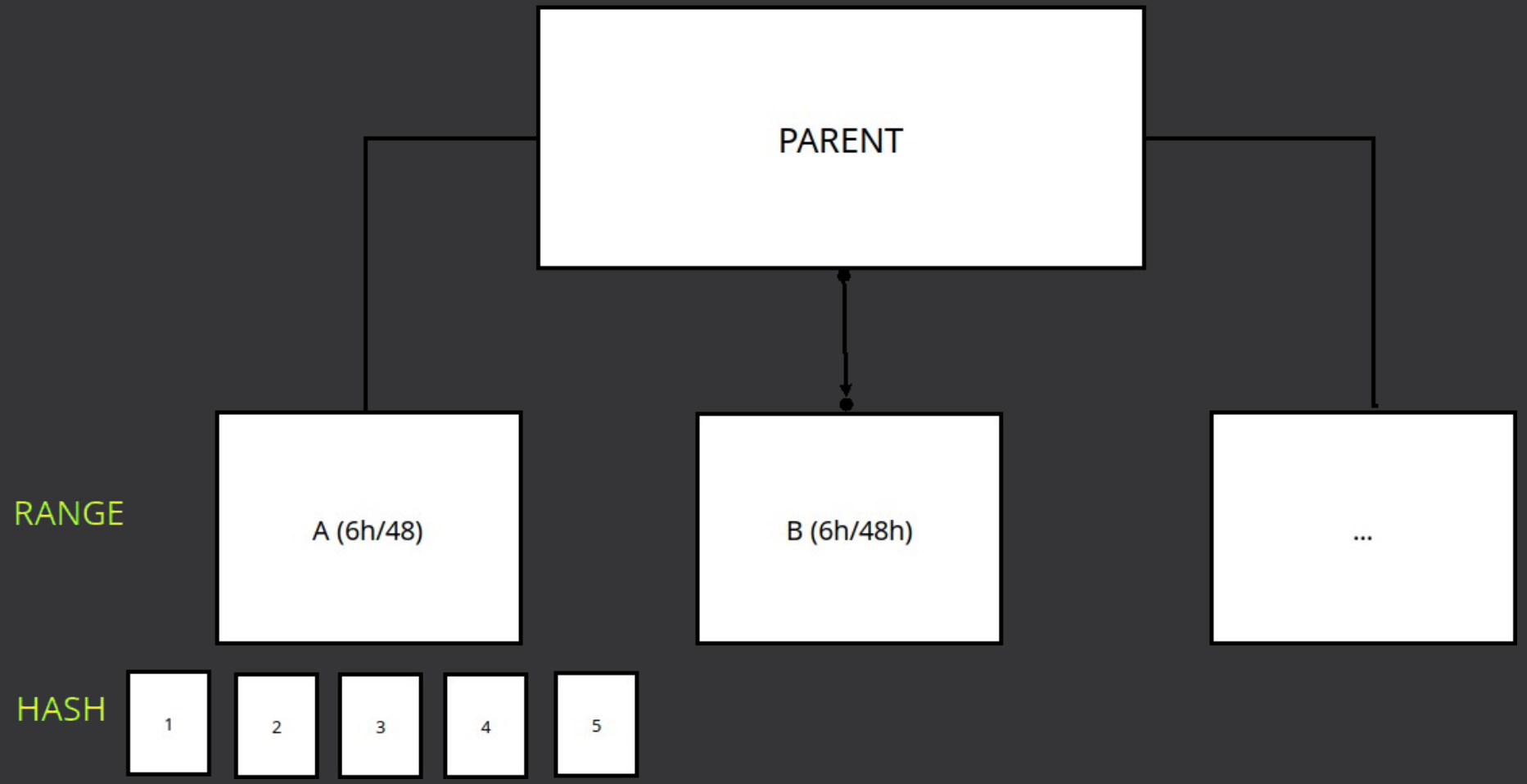
```
select * from set_chunk_time_interval('content_frames_video',interval '2 days');
select * from set_chunk_time_interval('content_frames_audio',interval '2 days');
```

Adding the HASH partition

```
select add_dimension('content_frames_video', by_hash('id', 5));
select add_dimension('content_frames_audio', by_hash('id', 5));
```


©2025 Percona

Timescale Partitions



©2025 Percona

Populate Tables

Simple bash scripts extracts and reformats the “points” into CSV format which can be directly uploaded to the postgres table content_frames_video.

```
#!/bin/bash
#
set -e
jq '.points.[]' video.json | jq --raw-output '@csv'
```

```
#!/bin/bash
#
set -e
jq '.points.[]' audio.json | jq --raw-output '@csv'
```

Import Points

```
psql db01_vectorscale <<_eof_

\copy content_frames_video (u_id,ts,frame) from program '/var/lib/postgresql/go_video.sh' with (FORMAT 'csv', DELIMITER ',', QUOTE '"')
\copy content_frames_audio (u_id,ts,frame) from program '/var/lib/postgresql/go_audio.sh' with (FORMAT 'csv', DELIMITER ',', QUOTE '"')

_eof_
```


©2025 Percona

Creating Indexes

Examples

```
-- L2 distance
CREATE INDEX ON items USING hnsw (embedding vector_l2_ops);

-- Inner product
CREATE INDEX ON items USING hnsw (embedding vector_ip_ops);

-- Cosine distance
CREATE INDEX ON items USING hnsw (embedding vector_cosine_ops);

-- L1 distance
CREATE INDEX ON items USING hnsw (embedding vector_l1_ops);

-- Hamming distance
CREATE INDEX ON items USING hnsw (embedding bit_hamming_ops);

-- Jaccard distance
CREATE INDEX ON items USING hnsw (embedding bit_jaccard_ops);
```

Supported types

vector - up to 2,000 dimensions
halfvec - up to 4,000 dimensions
bit - up to 64,000 dimensions
sparsevec - up to 1,000 non-zero elements

Index Options

m - the max number of connections per layer (16 by default)
ef_construction - the size of the dynamic candidate list for constructing the graph (64 by default)

Refer to github for more information

©2025 Percona

pgvectorscale:

build-time parameters

These parameters can be set when an index is created.

Parameter name	Description	Default value
`storage_layout`	`memory_optimized` which uses SBQ to compress vector data or `plain` which stores data uncompressed	memory_optimized
`num_neighbors`	Sets the maximum number of neighbors per node. Higher values increase accuracy but make the graph traversal slower.	50
`search_list_size`	This is the S parameter used in the greedy search algorithm used during construction. Higher values improve graph quality at the cost of slower index builds.	100
`max_alpha`	Is the alpha parameter in the algorithm. Higher values improve graph quality at the cost of slower index builds.	1.2
`num_dimensions`	The number of dimensions to index. By default, all dimensions are indexed. But you can also index less dimensions to make use of Matryoshka embeddings	0 (all dimensions)
`num_bits_per_dimension`	Number of bits used to encode each dimension when using SBQ	2 for less than 900 dimensions, 1 otherwise

©2025 Percona

Timescaledb configured to use a size of 6hr CHUNKS

```
-- Time: 2m:43s
create index on content_frames_video(id);
```

```
-- Time: 2m:11s
create index on content_frames_audio(id);
```

```
-- Time: 09h:26m
create index
    on content_frames_video
    using hnsw (frame vector_l2_ops)
    with (timescaledb.transaction_per_chunk, m = 16, ef_construction = 512);
```

```
-- Time: 13h:15m
create index
    on content_frames_audio
    using hnsw (frame vector_l2_ops)
    with (timescaledb.transaction_per_chunk, m = 16, ef_construction = 512);
```


©2025 Percona

Timescaledb configured to use a size of 48hr CHUNKS

```
-- Time: 60s
create index on content_frames_video(id);
```

```
-- Time: 45s
create index on content_frames_audio(id);
```

```
-- Time: 05h:55m
create index
    on content_frames_video
    using hnsw (frame vector_l2_ops)
    with (timescaledb.transaction_per_chunk, m = 16, ef_construction = 512);
```

```
-- Time: 7h:10m
create index
    on content_frames_audio
    using hnsw (frame vector_l2_ops)
```


©2025 Percona

The pgbench Workload

- A utility running bench-marks on PostgreSQL.
- User defined script runs the same sequence of SQL commands over and over,
- Can execute SQL statements in multiple concurrent database sessions, and then
- Calculates the average transaction rate (transactions per second) at the end of the workload.

©2025 Percona

Example pgbench Script

file: 01.sql

```
\set x01 random(1, 255)
\set x02 random(1, 255)
\set x03 random(1, 255)
\set x04 random(1, 255)
\set x05 random(1, 255)
.
.
.
\set x70 random(1, 255)
\set x71 random(1, 255)
\set x72 random(1, 255)
\set x73 random(1, 255)
\set x74 random(1, 255)
\set x75 random(1, 255)

-- <->: GET THE NEAREST DISTANCE, video
select frame <->
'[:x01,:x02,:x03,:x04,:x05,:x06,:x07,:x08,:x09,:x10,:x11,:x12,:x13,:x14,:x15,:x16,:x17,:x18,
:x19,:x20,:x21,:x22,:x23,:x24,:x25,:x26,:x27,:x28,:x29,:x30,
x31,:x32,:x33,:x34,:x35,:x36,:x37,:x38,:x39,:x40,:x41,:x42,:x43,:x44,:x45,:x46,:x47,:x48,:x4
9,:x50,:x51,:x52,:x53,:x54,:x55,:x56,:x57,:x58,:x59,
x60,:x61,:x62,:x63,:x64,:x65,:x66,:x67,:x68,:x69,:x70,:x71x72,:x73,:x74,:x75]' as "distance,
video"
from content_frames_video
order by 1
limit 1;
```


©2025 Percona

Example pgbench Script Invocation

```
pgbench -l --log-prefix=log-01 -n -d db01 -f 01.sql -t 50 -c 1 > results.log
```


©2025 Percona

Example pgbench Metrics Summary

```
pgbench (17.4 (Ubuntu 17.4-1.pgdg24.04+2))
transaction type: 03.content_frames_video-pgvectrscale_01-Cosine.sql
scaling factor: 1
query mode: simple
number of clients: 1
number of threads: 1
maximum number of tries: 1
duration: 900 s
number of transactions actually processed: 53523
number of failed transactions: 0 (0.000%)
latency average = 16.815 ms
initial connection time = 12.320 ms
tps = 59.470623 (without initial connection time)
```


©2025 Percona

Analysis & Conclusion

What was figured out.

Performance Metrics

CHUNK Size: 6hrs

TABLE	Number Partitions	Avg Table Size, MB	Avg Index Size, MB	Avg Total Size, MB
table_abc	325	115	196	310
table_abc	310	66	152	218

CHUNK Size: 48hrs

TABLE	Number Partitions	Avg Table Size, MB	Avg Index Size, MB	Avg Total Size, MB
table_def	45	827	1412	2239
table_def	40	514	1175	1689

©2025 Percona

Performance Metrics (*partial*), Averaged Over Several Runs of 15 minutes duration

CHUNK, hrs	Table	DML operation	latency average, ms	latency stddev, ms	TPS
6	table_abc	SELECT	46,978.76	27,746.94	0.11
6	table_def	SELECT	46,536.46	35,260.77	0.10
6	table_abc	INSERT	9.04	3.46	217.03
6	table_def	INSERT	7.49	2.74	217.03
6	table_abc	DELETE	14.98	1.62	217.67
6	table_def	DELETE	14.53	1.90	217.67
48	table_abc	SELECT	2,154.98	1,102.09	3.02
48	table_def	SELECT	1,152.46	397.63	2.98
48	table_abc	INSERT	11.46	4.03	402.21
48	table_def	INSERT	9.32	3.21	401.80
48	table_abc	DELETE	2.11	0.80	403.19
48	table_def	DELETE	1.96	0.66	402.65

©2025 Percona

Observations

The larger the CHUNK:
- fewer partitions
- SELECTs are faster

Hash Partitions vs Range Partitions
- more partitions
- INSERTs are faster
- SELECTs are slower

INDEXes
- peformance varied
    - pgvector vs pgvectorscale
    - creation times
    - faster INSERTs vs slower SELECTs

©2025 Percona

Moving Forward: a proposed architecture

A two node cluster replication, via logical replication, is recommended:

- Node 1: PUBLISHER
    - to be used as the SOURCE host for all DML operations
    - to be partitioned using timescale by range based upon time.
    - each ranged partitioned to be further subdivided by hash

- Node 2: SUBSCRIBER
    - Partition numbers are fewer than PUBLICATION node
    - Partition CHUNKs are to be optimized to fit system RAM
    - Data to be organized in partitions per algorithmically calculations

©2025 Percona

CAVEAT

Keep in mind the following factors affecting performance when using the vector based indexes:

- type of index parameter defined
- number of dimensions used for each index
- number partitions in the database
- the system load
- database life cycle i.e. size and growth rate over time

*Remember: both pgvector and pgvectorscale indexes are undergoing **constant** development.*

©2025 Percona

PERCONA®

PERCONA
for PostgreSQL

Thank You :-)

©2025 Percona

PERCONA®

PERCONA
for PostgreSQL

Questions?

©2025 Percona